

Assignment 2: Threads, Scheduling, Synchronization

Suhaib Abdi Muhummed, muhummed@kth.se

Abdulaziz Hassan Farah, ahfarah@kth.se

Daniel Sherif, dsherif@kth.se

Question 1: Scheduling Algorithms

Consider the following set of processes, with the length of the CPU burst is given in milliseconds. A smaller priority number implies a higher priority.

Process	Arrival Time	Burst Time	Priority
P1	0	8	4
P2	1	3	3
P3	2	5	2
P4	8	1	0

Figure 1: Enter Caption

For each of FCFS, SJF, preemptive SRTF, non-preemptive priority scheduling, and Round-Robin (quantum = 4):

1. Draw the Gantt chart to illustrate the execution of these schedule.
2. What is the turnaround time of each process for each of the scheduling algorithms?
3. What is the waiting time of each process for each of these scheduling algorithms?
4. Which algorithm results in the minimum average waiting time?

Answers:

Based on the waiting times calculated for all algorithms, the one that gives the minimum average waiting time is preemptive SRTF. This happens because SRTF always selects the process with the shortest remaining burst time, so long processes keep getting preempted whenever a shorter job arrives. As a result, short processes wait very little, and the overall average waiting time becomes the smallest. In this case, the average waiting time is 3 seconds, which is the lowest among all the algorithms.

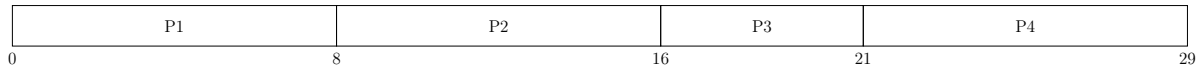


Figure 2: Gantt chart for FCFS

Process	Arrival	Burst	Turnaround	Waiting
P1	0	8	8	0
P2	1	7	16	9
P3	2	5	14	9
P4	8	1	9	8

Table 1: Turnaround and waiting times for FCFS

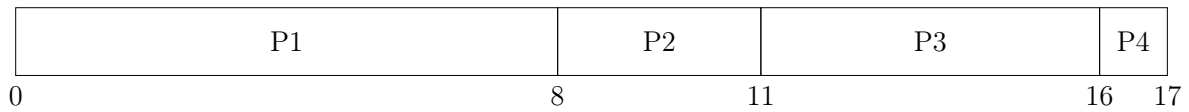


Figure 3: Gantt chart for SJF

Process	Arrival	Burst	Turnaround	Waiting
P1	0	8	8	0
P2	1	3	11	8
P3	2	5	15	10
P4	8	1	1	0

Table 2: Turnaround and waiting times for SJF

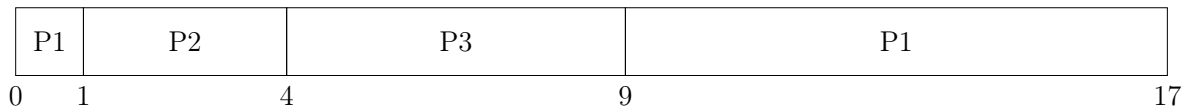


Figure 4: Gantt chart for Preemptive SRTF

Process	Arrival	Burst	Turnaround	Waiting
P1	0	8	17	9
P2	1	3	3	0
P3	2	5	7	2
P4	8	1	2	1

Table 3: Turnaround and waiting times for SRTF

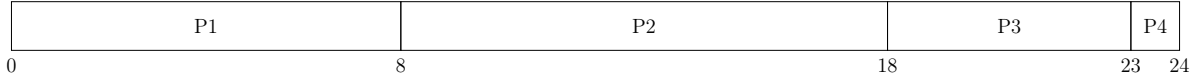


Figure 5: Gantt chart for Non-preemptive Priority

Process	Arrival	Burst	Turnaround	Waiting
P1	0	8	17	9
P2	1	3	9	6
P3	2	5	5	0
P4	8	1	1	0

Table 4: Turnaround and waiting times for Non-preemptive Priority

Question 2: Scheduler Criteria

Provide one concrete example for each case, to illustrate the scenarios when the following scheduling criteria conflict:

1. Average turnaround time and Maximum waiting time
2. Response time and Maximum waiting time
3. System Job Throughput and Response Time

Answer:

1. Average turnaround time and Maximum waiting time

We choose the preemptive SRTF algorithm because it gives the smallest average turnaround time. But this also means that the waiting time for some processes becomes long. For example, p1 arrives at time 0 and starts running, but when p2 arrives at time 2, it takes over the CPU because p2 has a shorter burst time than p1 ($6 < 8$). Then p2 keeps running, and when p3 arrives at time 4, p2 still has a shorter remaining burst time than p3 ($4 < 7$), so p2 continues until it finishes.

After that, p3 runs, and when p3 is done, p1 finally continues and runs until it completes. In this case, p1 gets a long waiting time (13 seconds), while the average turnaround time becomes smaller.

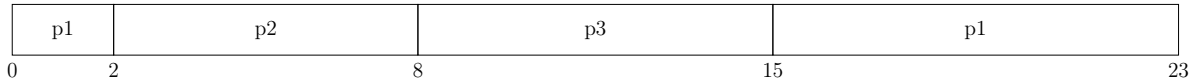


Figure 6: Gantt chart for SRTF

Process	Arrival	Burst	Completion	Turnaround	Waiting
p1	0	10	23	23	13
p2	2	6	8	6	0
p3	4	7	15	11	3

Table 5: Turnaround and waiting times for SRTF

2. Response time and Maximum waiting time

Response time = first time the process gets the CPU – Arrival time.

In Round Robin (quantum = 4), the CPU is shared so that every process gets a small amount of CPU time early. This means many processes receive their first CPU access quickly, which reduces the response time. However, because processes are interrupted often and put back into the queue, the waiting time becomes longer, especially for processes with long burst times. They have to wait through several cycles before they finish. This shows Round Robin reduces response time, but it increases the maximum waiting time.

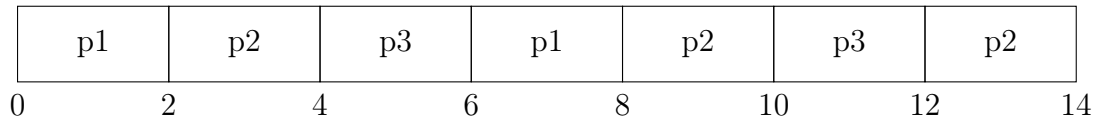


Figure 7: Gantt chart for Round Robin (quantum = 4)

Process	Arrival	Burst	Completion	Turnaround	Waiting
p1	0	8	16	16	8
p2	2	10	28	26	16
p3	4	6	24	20	14

Table 6: Response and waiting times for Round Robin

3. System Job Throughput and Response Time

SJF completes many short burst-time processes quickly, which increases the system job throughput.

However, the downside is that long processes get a much longer response time, because they have to wait until all the shorter processes finish first.

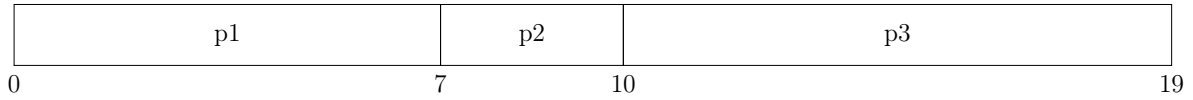


Figure 8: Gantt chart for SJF

Process	Arrival	Burst	Completion	Turnaround	Waiting
p1	0	7	7	7	0
p2	2	3	10	8	5
p3	4	9	19	15	6

Table 7: Response and waiting times for SJF

Question 3: Process Creation.

Recall that a resource-allocation graph without cycles has no deadlock, and a resource-allocation graph with cycles may or may not have deadlocks. In the following resource-allocation graphs:

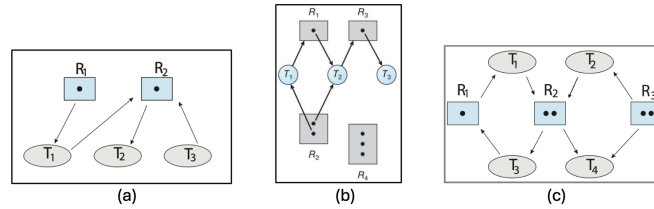


Figure 9: Resource-allocation graphs for Question 3

Answer:

1. A.

One instance of R_1 is assigned to T_1 . T_1 is requesting one instance of R_2 . One instance of R_2 is assigned to T_2 . T_3 is requesting one instance of R_2 .

B.

T_1 has one instance of R_2 assigned to it and is requesting one instance of R_1 . T_2 has one instance of R_1 and one instance of R_2 assigned to it, and is requesting one instance of R_3 . T_3 has one instance of R_3 assigned to it.

C.

T_1 has one instance of R_1 assigned and is requesting one instance of R_2 . T_2 is requesting one instance of R_2 and has one instance of R_3 assigned to it. T_3 is requesting one instance of R_1 and has one instance of R_2 assigned to it. T_4 has one instance of R_2 assigned and one instance of R_3 assigned to it.

2. Deadlock Analysis:

Graph A and Graph B have no deadlocks, because there are no cycles.

A.

T2 releases R2, and T3 picks up R2. T1 finishes with R1 and waits for T3 to release R2. T1 then acquires the single instance of R2 and finishes. Execution complete.

B.

T1 finishes with R2 and waits for T2 to release R1. T2 finishes with R1 and continues with its already acquired instance of R2. T1 picks up R1. T3 finishes with the single R3 instance. T2 finishes with the second R2 instance, acquires R3, and completes.

3.

Graph C possibly has a deadlock due to the cycle:

$$T1 \rightarrow R2 \rightarrow T3 \rightarrow R1 \rightarrow T1$$

This is because resources R1 and R2 have all their instances allocated. Resource R1 has only one instance. Resource R2 has two instances, but one is already allocated to T4, leaving only one instance available for the cycle. Therefore, the cycle is problematic.

Question 4: Banker's Algorithm.

Given the following system snapshot:

Processes	Allocation				Max				Available			
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

Figure 10: System snapshot for Question 4

Answer:

1. **List the number of instances per resource. List down the need matrix.**

The total number of instances per resource is computed as follows:

$$A : 1 + (0 + 1 + 1 + 0 + 0) = 3$$

$$B : 5 + (0 + 0 + 3 + 6 + 0) = 14$$

$$C : 2 + (1 + 0 + 5 + 3 + 1) = 12$$

$$D : 0 + (2 + 0 + 4 + 2 + 4) = 12$$

Therefore, the total number of resources is:

$$\{3, 14, 12, 12\}$$

The Need matrix is given by:

$$\text{Need} = \text{Max} - \text{Allocation}$$

Process	A	B	C	D
P0	0	0	0	0
P1	0	7	5	0
P2	1	0	0	2
P3	0	0	2	0
P4	0	6	4	2

2. Can P1 request (0,4,2,0)?

Since (0, 4, 2, 0) is less than or equal to (0, 7, 5, 0), which comes from the Need matrix, and (0, 4, 2, 0) is also less than or equal to (1, 5, 2, 0), which represents the available resources, the request can indeed be granted immediately.

3. Run Banker's safety check. Is the system safe?

By adding the available resources to the released resources, we can iteratively find a safe order in which all processes can complete.

$$P0 \text{ need} \leq (1, 5, 2, 0)$$

$$\Rightarrow \text{Work} = (1, 5, 2, 0) + P0_{\text{alloc}} = (1, 5, 2, 0) + (0, 0, 1, 2) = (1, 5, 3, 2)$$

Question 5: Minimum resources.

Given the following system snapshot:

Processes	Allocation				Max				Available			
P0	1	0	1	2	1	0	1	2	0	x	4	1
P1	1	0	3	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	2	3	2	0	6	5	2				

Figure 11: Enter Caption

Determine the smallest value of x for which this is a safe state. Briefly explain how you derive the answer. Hint: Compute the need matrix first and then identify the sequence of process that could complete if x is large.

Answer:

The given system snapshot is:

$$\text{Available} = (0, x, 4, 1)$$

$$\text{Allocation} = \begin{bmatrix} 1 & 0 & 1 & 2 \\ 1 & 0 & 3 & 0 \\ 1 & 3 & 5 & 4 \\ 0 & 2 & 3 & 2 \end{bmatrix} \quad \text{Max} = \begin{bmatrix} 1 & 0 & 1 & 2 \\ 1 & 7 & 5 & 0 \\ 2 & 3 & 5 & 6 \\ 0 & 6 & 5 & 2 \end{bmatrix}$$

1. Compute the Need matrix

$$\text{Need} = \text{Max} - \text{Allocation}$$

$$\text{Need} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 7 & 2 & 0 \\ 1 & 0 & 0 & 2 \\ 0 & 4 & 2 & 0 \end{bmatrix}$$

2. Safety check (Banker's Algorithm)

Initial available resources:

$$\text{Available}_0 = (0, x, 4, 1)$$

Step 1: P0 can run since $\text{Need}_0 = (0, 0, 0, 0)$. After P0 finishes:

$$\text{Available}_1 = (1, x, 5, 3)$$

Step 2: P2 can run because

$$(1, 0, 0, 2) \leq (1, x, 5, 3)$$

After P2 finishes:

$$\text{Available}_2 = (2, x + 3, 10, 7)$$

Step 3: P3 can run only if

$$(0, 4, 2, 0) \leq (2, x + 3, 10, 7) \Rightarrow x \geq 1$$

After P3 finishes:

$$\text{Available}_3 = (2, x + 5, 13, 9)$$

Step 4: P1 can run only if

$$(0, 7, 2, 0) \leq (2, x + 5, 13, 9) \Rightarrow x \geq 2$$

Thus the system is safe if and only if $x \geq 2$.

Safe sequence for $x=2$:

$$\langle P_0, P_2, P_3, P_1 \rangle$$

Final Answer:

The smallest value of x that yields a safe state is $x = 2$.

Programming Section:

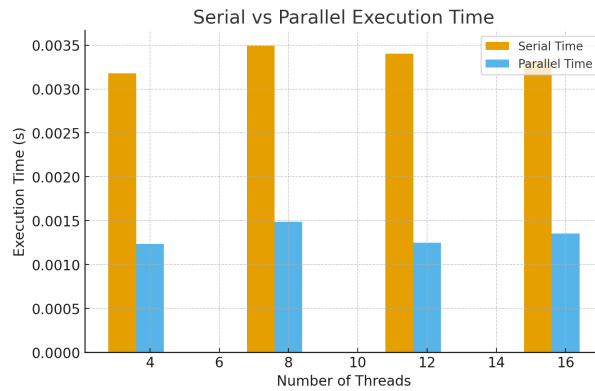
Question 6: Thread creation, termination, and synchronization

This is a programming assignment. Write a C program by modifying the source code skeleton pthread_sum.c to use a group of threads to parallelize the sum of an array. Submit your codes and Makefile in a zip file.

- (a) The master thread performs a Serial Sum first. It creates an array of 1,000,000 random floats in $[0,1]$, and prints out the sum of the array.
- (b) Then, a group of worker threads are created. Each thread is assigned with a unique ID in $[0, \text{num_threads} - 1]$, where num_threads is given by argv[1]
- (c) Modify the code so that each thread sums up a portion of the array, and a final result is aggregated by the master thread from all threads' results and print out. Briefly describe your design of work distribution among threads.
- (d) Add 2 timers. One measures the time spent in the serial sum and one measures the parallel sum.
- (e) Run your program with 4,8,12,16 threads. Please include screenshots of your output here.
- (f) Make a bar chart to visualize your results. Analyze your results.

Answer: The code solution .

Across all configurations (4, 8, 12, 16 threads), the parallel version is consistently faster than the serial version. Parallel times range from **0.00123 - 0.00149 seconds**, compared to **0.0032 - 0.0035 seconds** for serial execution. This corresponds to a speedup of roughly 2.4x to 2.7x improvement This is a strong result, especially for a memory-bound task.



However, the performance does *not* scale linearly with the number of threads. For example:

- 4 threads -> **best performance**
- 8 threads -> slight increase in parallel time
- 12/16 threads -> **no further improvement**

This diminishing return is expected and occurs due to several factors:

a) Thread creation & management overhead

More threads =

- More scheduling
- More context switches
- More coordination overhead

At some point, the overhead outweighs the benefit of parallelism.

b) Memory bandwidth limitations Summing an array is memory-bound, not compute-bound:

- All threads simultaneously read from memory
- DRAM bandwidth becomes saturated
- Adding threads doesn't make memory faster

So performance becomes limited by the memory subsystem.

c) CPU cache contention More threads means more cache lines being evicted:

- Threads compete for L1/L2/L3 cache
- False sharing / cache thrashing may occur
- Data movement dominates computation

This directly reduces speedup when threads exceed available CPU cores.

Question 7: multi-threaded histogram

This is a programming assignment. Write a C program to create a group of threads to parallelize building the histogram of an array. You should use 30 bins for the histogram. You can reuse your code in Question 3. Submit your codes and Makefile in a zip file.

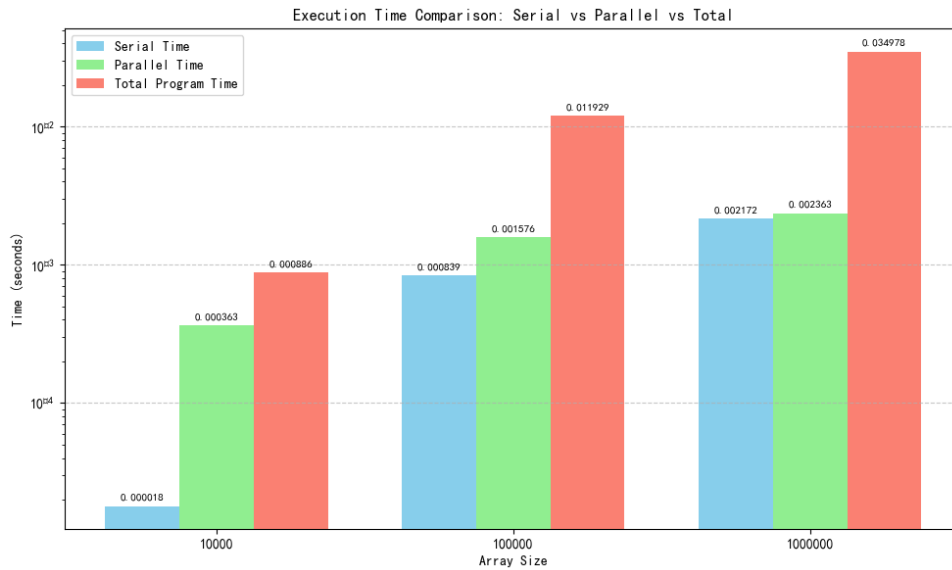
- (a) Modify the code so that it creates an array of doubles, the array length is given by `argv[2]`. The array should be initialized with random values in the range `[0.0, 1.0]`.
- (b) Serial histogram: add code to create the histogram of the array. Print out the histogram and time.
- (c) Parallel histogram: Modify the thread function so that each thread builds the histogram of a portion of the array, and a final histogram is aggregated from all threads' results. Print out the histogram and time.
- (d) Briefly describe your design of shared data structures and synchronization.
- (e) Run your code: `./program 3 10000`, `./program 3 100000`, and `./program 3 1000000`. Please include screenshots of your output here.
- (f) Plots that visualize the output histogram. Analyze your results.

Answer: The code solution

We used a shared data structure to minimize contention and avoid synchronization overhead during parallel execution. Instead of having all threads update a single global histogram, which would require mutex locks, the design uses **per-thread local histograms** stored in a 2D array (`local_histograms[thread_id][bin]`). Each thread processes its own segment of the input array independently and writes only to its private histogram, thereby eliminating the need for locking during the counting phase.

After all threads finish, the main thread performs a final **aggregation step**, summing the local histograms into the shared global histogram (`global_histogram`). This approach ensures safe parallelism without explicit synchronization primitives, reduces contention, and keeps the parallel section fully lock-free. However, it also introduces some trade-offs, including extra memory usage for per-thread histograms, a serial reduction step that cannot be parallelized, and thread-creation overhead that can dominate when the problem size is small.

These effects are clearly visible in the performance results: across all tested input sizes, the serial histogram outperforms the parallel version. For 10,000, 100,000, and 1,000,000 elements, the serial times are 0.000018s, 0.000839s, and 0.002172s respectively, while the parallel version takes longer at 0.000363s, 0.001576s, and 0.002363s.



The reason for the slower parallel performance is the fixed overhead associated with creating and joining threads, initializing per-thread histograms, and performing the final serial merge stage. As the array size grows, the gap between serial and parallel execution narrows because the actual computation grows while the overhead remains constant.

Question 8: Lock-free synchronization

This is a programming assignment. Compare_and_swap (CAS) instruction leverages hardware atomics on today's computers and can be used to design lock-free data structures. Write a C program by modifying the source code skeleton `pthread_stack.c` to implement Stack as a linked list of Nodes and each thread can perform `push()` and `pop()` on it. Submit your codes and Makefile in a zip file.

- Each thread performs two functions, `push()` for adding a node to the linked list of nodes, and `pop()` for removing the top node from the linked list. The code should print out the number of nodes remaining in Stack.
- In `pop()` and `push()`, add the implementation based on mutex locks to update top.
- Briefly describe the global data structures in your code and how are access synchronized on them.
- Run your program with 1, 2, 4, 8, 16, 32 threads. Please include screenshots of your output here.
- In `pop()` and `push()`, add the implementation based on compare-and-swap (CAS) operations to update top.
- Briefly describe the global data structures in your code and how are access synchronized on them.
- Run your program with 1, 2, 4, 8, 16, 32 threads. Please include screenshots of your output here.

Answer: The code solution

We use several global data structures that are shared between threads. The `node_id_counter` variable provides unique IDs for newly created nodes and is protected by the `counter_mutex` to ensure that increments are performed atomically. The global stack is represented by the pointer `top`, which always refers to the first node in the linked list. Access to this shared stack is synchronized in two different ways depending on the version being executed:

- in the **mutex lock** version, all push and pop operations lock the `stack_mutex`, ensuring mutual exclusion when modifying the `top` pointer.
- in the **Compare-and-Swap (CAS)** version, the stack is updated using the atomic `compare-and-swap` operation, which provides lock-free synchronization by ensuring that a thread only updates `top` if no other thread has changed it.

Other global variables such as `num_threads` are initialized before thread creation and are read only thereafter, so they do not require synchronization. Overall, mutex locking provides blocking synchronization, forcing threads to wait for access, while CAS provides non-blocking synchronization, where threads repeatedly retry the operation until the atomic update succeeds. This avoids blocking but introduces extra overhead when contention is high due to repeated retries.

Mutex Version	CAS Version
Number of threads: 2	Number of threads: 2
Starting Mutex version	Starting CAS version
Thread 0 starting	Thread 0 starting
Thread 1 starting	Thread 1 starting
Pushed node 0	Pushed node 1
Pushed node 1	Pushed node 2
Pushed node 2	Popped node 2
Pushed node 3	Popped node 1
Popped node 3	Pushed node 3
Popped node 1	Thread 1: exit
Pushed node 4	Pushed node 0
Thread 1: exit	Pushed node 4
Popped node 2	Popped node 4
Popped node 4	Popped node 3
Pushed node 5	Pushed node 5
Thread 0: exit	Thread 0: exit
Mutex: Remaining nodes	CAS: Remaining nodes
Mutex: Remaining nodes in stack:	CAS: Remaining nodes in stack:
Node ID: 5	Node ID: 5
→ Node ID: 0	→ Node ID: 0
Total remaining nodes: 2	Total remaining nodes: 2

Table 8: Comparison of Mutex vs. CAS Execution Output (2 Threads)

The experimental results clearly highlight the behavioral differences between the two approaches. With only 1 and 2 threads, both mutex and CAS methods behave identically because there is no contention; the operations occur sequentially, leaving exactly one node in the stack. As the number of threads increases to 4, 8, 16, and 32, the mutex-based implementation continues to produce stable and predictable results: each thread contributes exactly one remaining node. This is expected, since the mutex enforces strict serialization, preventing any race conditions and ensuring that every push and pop occurs in a deterministic order.

The CAS-based implementation behaves similarly at lower thread counts, but at higher levels of concurrency small inconsistencies start to appear in the ordering of the final nodes. For example, the presence of out-of-order or unexpected node IDs when running with 16 or 32 threads. These inconsistencies occur because CAS relies on retry loops and does not block threads; under heavy contention, many CAS attempts fail, leading to more complex and less predictable interleavings of operations.

Despite this, the total number of remaining nodes is always correct, demonstrating that the CAS implementation is functionally sound but more sensitive to timing.

In summary, mutex locking provides deterministic behavior but scales poorly under contention, while CAS avoids blocking and scales better but results in less predictable execution patterns—exactly the trade-off expected between lock-based and lock-free concurrency.